

DevOps Lab App

Repositorio: https://github.com/santilp95/01_devops_cd_cd

Integrantes

- Jeisson Alejandro Fuquene Buitrago (jeissonfubu@unisabana.edu.co)
- Santiago López Amaya (santiagoloam@unisabana.edu.co)

Aplicación web mínima (**Node.js + TypeScript + Express**) usada como caso práctico para el laboratorio técnico de la Actividad 3: **configurar un pipeline CI/CD básico usando GitHub Actions y Jenkins.**

Endpoints

Método	Ruta	Descripción
GET	/health	Health check, responde { "status": "ok" }
GET	/api/greet?name=X	Devuelve un saludo personalizado
POST	/api/sum	Recibe { "a": number, "b": number } y devuelve la suma

Ejecutar en local

```
npm install
npm run typecheck    # verificación de tipos (tsc --noEmit)
npm run lint         # ESLint (reglas TS)
npm test             # pruebas con Jest + ts-jest + cobertura
npm run build        # compila TypeScript a dist/
npm start            # corre dist/index.js en http://localhost:8080
npm run dev          # alternativa: corre src/index.ts directo con ts-node
```

Flujo CI/CD

CI — GitHub Actions (.github/workflows/ci.yml)

Se dispara automáticamente en cada push y en cada pull_request hacia main. Etapas:

1. Checkout del código (actions/checkout).
2. Configuración de Node.js con caché de dependencias.
3. Instalación de dependencias (npm ci).
4. Verificación de tipos (npm run typecheck).
5. Análisis estático con ESLint (npm run lint).
6. Ejecución de pruebas con Jest (npm test) y publicación del reporte de cobertura como artefacto.
7. Compilación a JavaScript (npm run build), validando que el proyecto compile antes de construir la imagen.

Si cualquier paso falla, el workflow se marca en rojo y bloquea la confianza en ese commit/PR.

Resumen del run — commit disparador, estado Success y artefacto generado:

Todos los steps del job, en el mismo orden que la lista de arriba:

Paso 1 — Checkout del código:

Pasos 2 a 4 — Configurar Node.js, instalar dependencias y verificación de tipos:

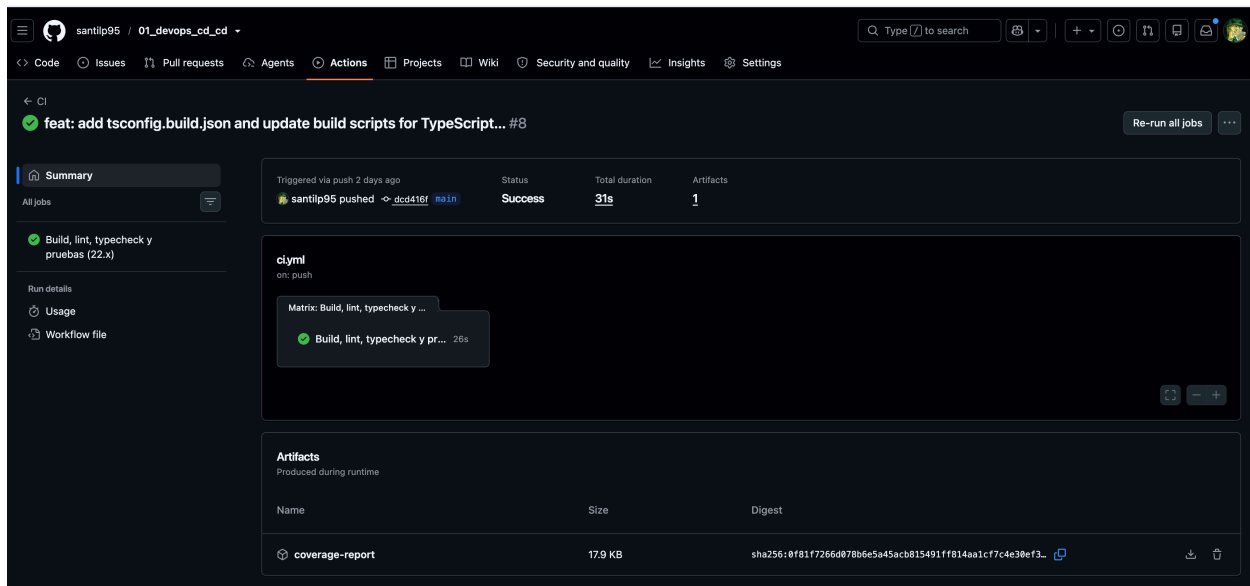


Figure 1: Resumen del pipeline de CI

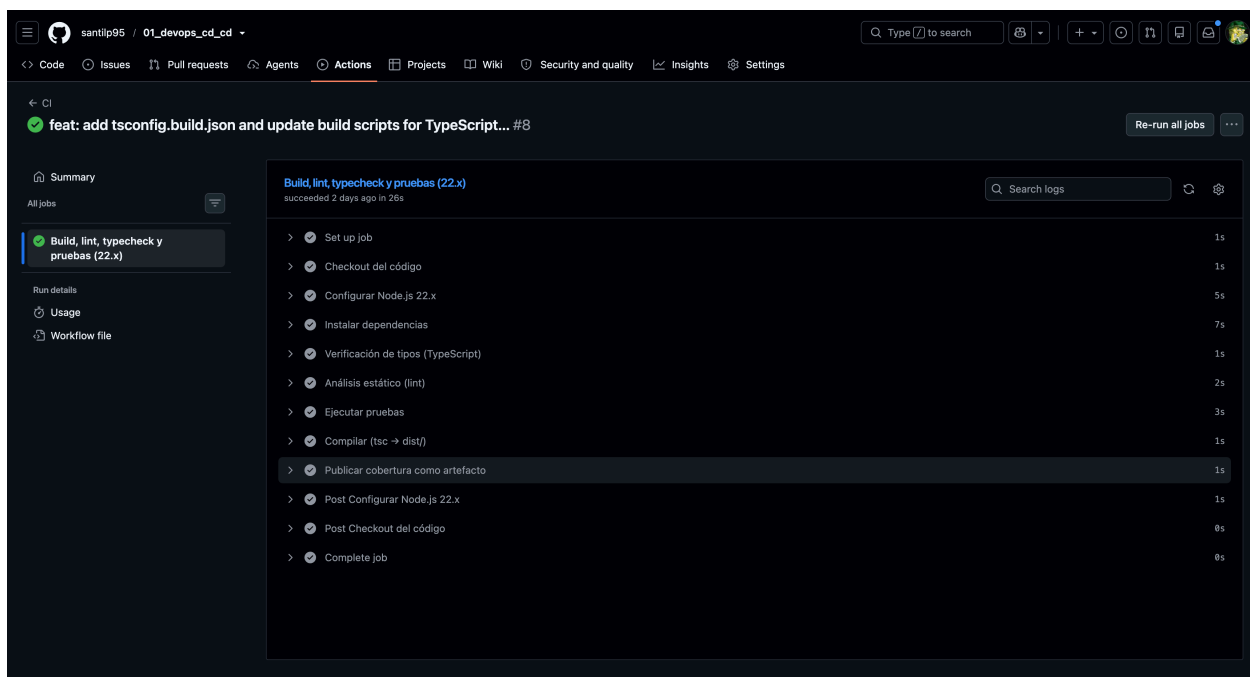


Figure 2: Lista completa de steps del job

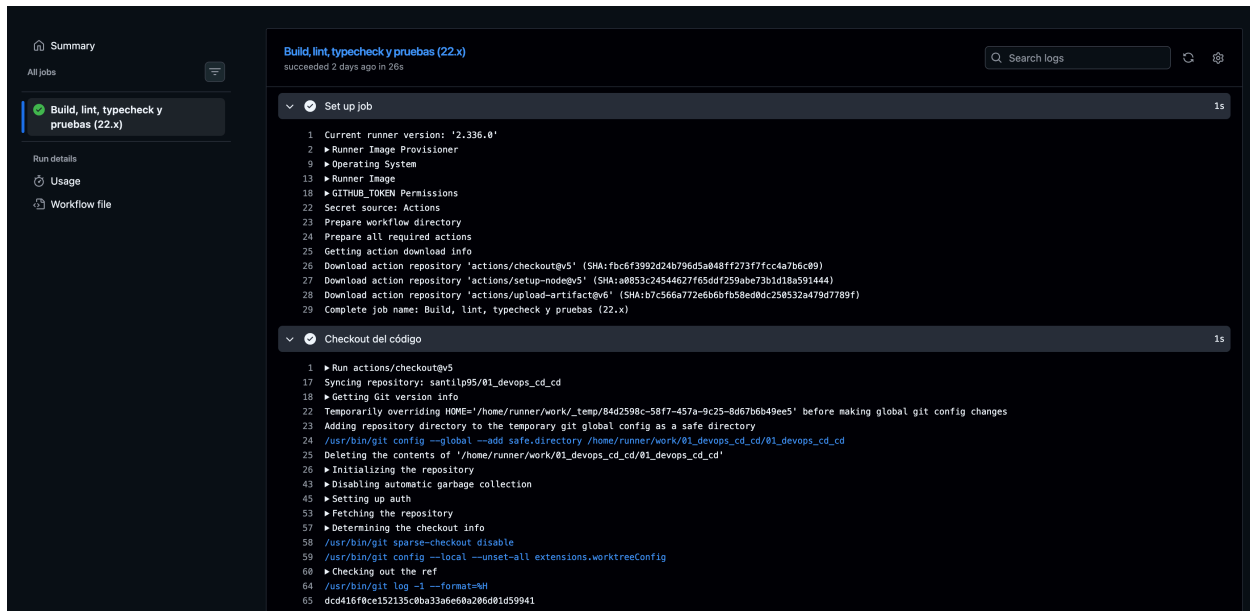


Figure 3: Checkout del código

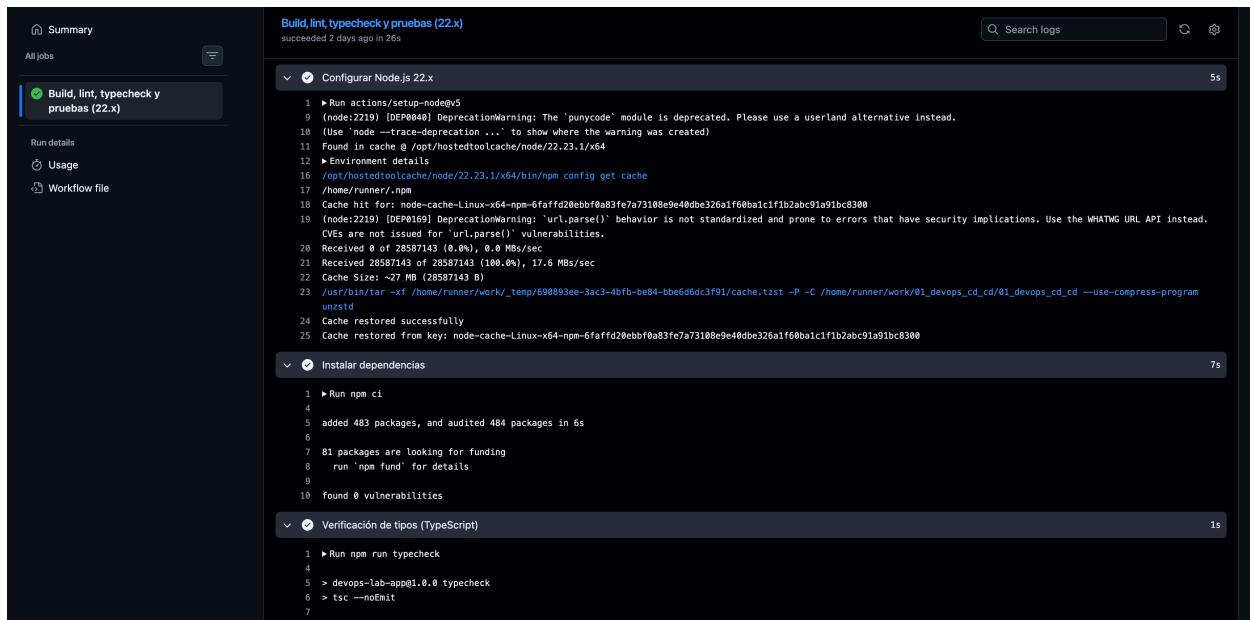


Figure 4: Configurar Node.js, instalar dependencias y typecheck

Pasos 5 y 6 — Análisis estático (lint) y ejecución de pruebas (7 tests en verde, 100% de cobertura):

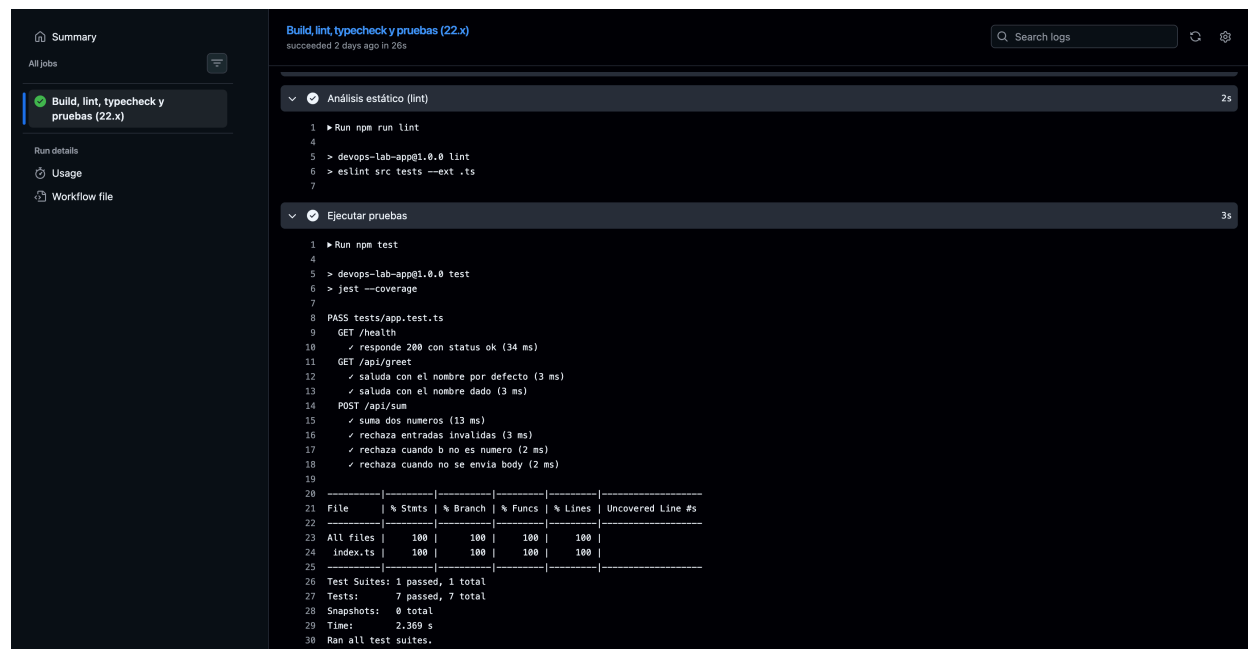


Figure 5: Lint y ejecución de pruebas

Paso 7 — Compilación (tsc → dist/) y publicación de la cobertura como artefacto:
Cierre del job — artefacto subido y limpieza final (Post steps):

CD — GitHub Actions (.github/workflows/deploy.yml)

Se dispara automáticamente vía workflow_run cuando el workflow **CI** termina en la rama main, y solo si concluyó con éxito (conclusion == 'success'). Etapas:

1. Checkout del commit exacto que probó el CI (head_sha, no el último de main).
2. Autenticación contra GCP con **Workload Identity Federation** (sin llaves JSON, usa WIF_PROVIDER + WIF_SERVICE_ACCOUNT).
3. Login a Artifact Registry con el access token obtenido.
4. Build y push de la imagen a us-central1-docker.pkg.dev/<PROJECT_ID>/mi-app/mi-app:<sha>.
5. Deploy a Cloud Run (servicio mi-app, región us-central1) con la acción deploy-cloudrun.

Requiere estos secrets configurados en el repositorio de GitHub (**Settings** → **Secrets and variables** → **Actions**): GCP_PROJECT_ID, WIF_PROVIDER, WIF_SERVICE_ACCOUNT.

CD — Jenkins (Jenkinsfile)

Pipeline declarativo con solo 3 stages (sin pruebas de integración, esas ya las cubre el CI de GitHub Actions). Es una ruta de despliegue alternativa/manual al mismo servicio de Cloud Run, pensada para el laboratorio de Jenkins:

1. **Checkout:** clona el repositorio (checkout scm).

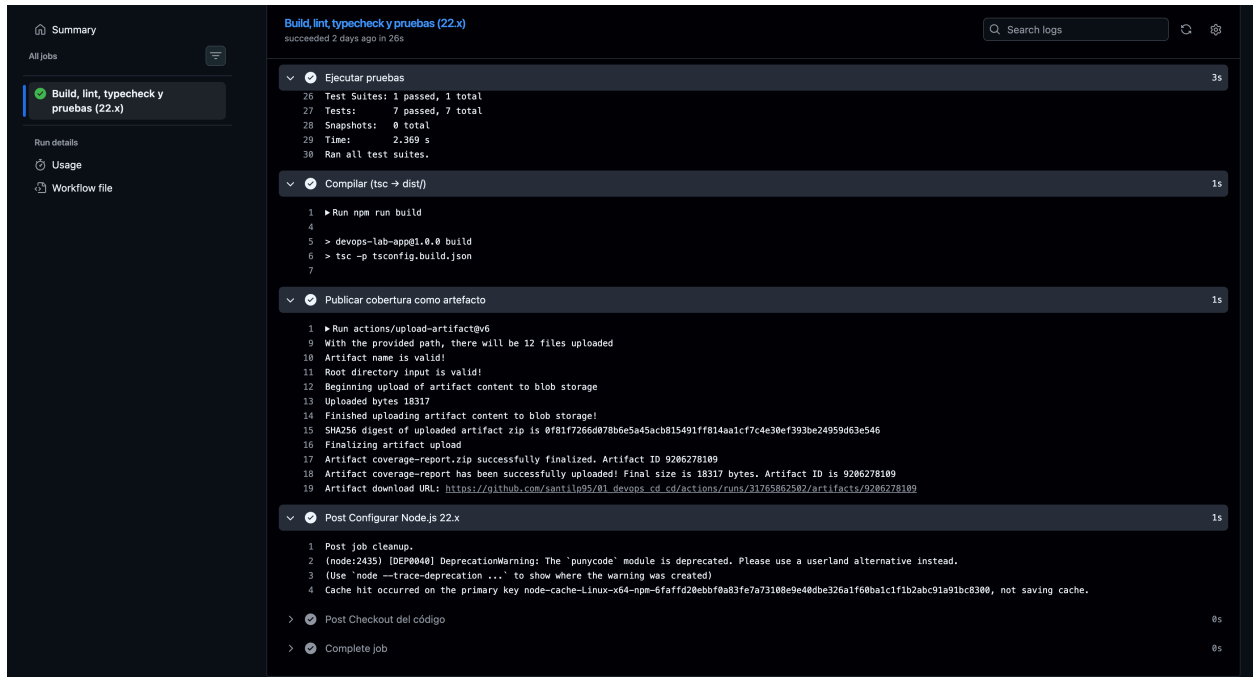


Figure 6: Compilación y publicación de cobertura

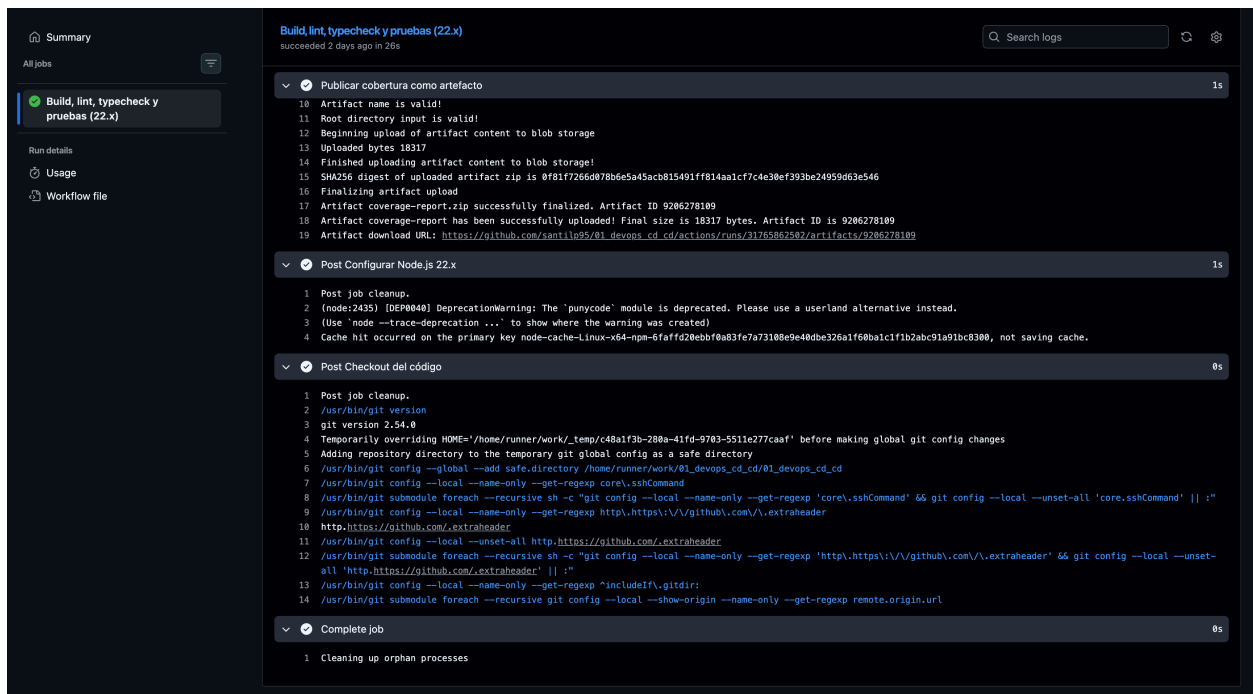


Figure 7: Cierre del job

2. **Build:** construye la imagen Docker (multi-stage — compila TypeScript y produce una imagen final mínima corriendo como usuario no root) y la etiqueta con el número de build y latest.
3. **Deploy:** se autentica en GCP con una service account (clave JSON, distinto método de auth al de GitHub Actions), hace push de la imagen a **Artifact Registry** y despliega el servicio en **Cloud Run** con `gcloud run deploy`.

Se dispara automáticamente con cada push a main mediante `pollSCM('H/5 * * * *')`; Jenkins revisa el repositorio cada 5 minutos y arranca el build si hay commits nuevos en la rama configurada en el job. Se usa *polling* en vez de un webhook porque este Jenkins corre local (no accesible desde internet); si se expone públicamente, se puede cambiar a `githubPush()` + webhook de GitHub para un trigger inmediato.

Importante: Jenkinsfile y `deploy.yml` publican en el **mismo** repositorio de Artifact Registry (mi-app) y despliegan al **mismo** servicio de Cloud Run (mi-app, us-central1) — es intencional, para reusar la infraestructura ya creada por el flujo de GitHub. Ambos ahora reaccionan a un push a main (uno vía `workflow_run` casi inmediato, el otro vía poll con hasta 5 min de latencia), así que un mismo commit puede disparar los dos pipelines; no hay conflicto, pero el último en terminar es el que queda como revisión activa en Cloud Run.

Resumen del pipeline — job `deploy-cloud-run-pipe`, run #6, los 4 stages (Checkout, Build, Deploy, Post Actions) en verde:

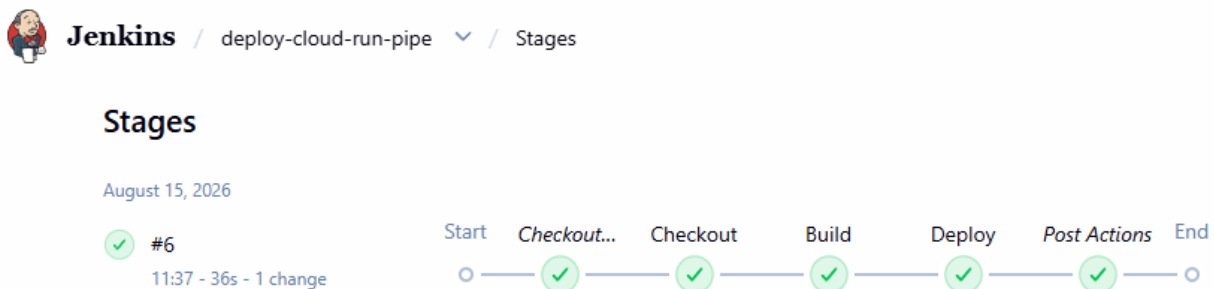


Figure 8: Stages del pipeline de Jenkins

Credencial configurada en Jenkins — la clave de la service account de GCP guardada como *Secret file* (`gcp-service-account-key`), la misma que referencia el Jenkinsfile:

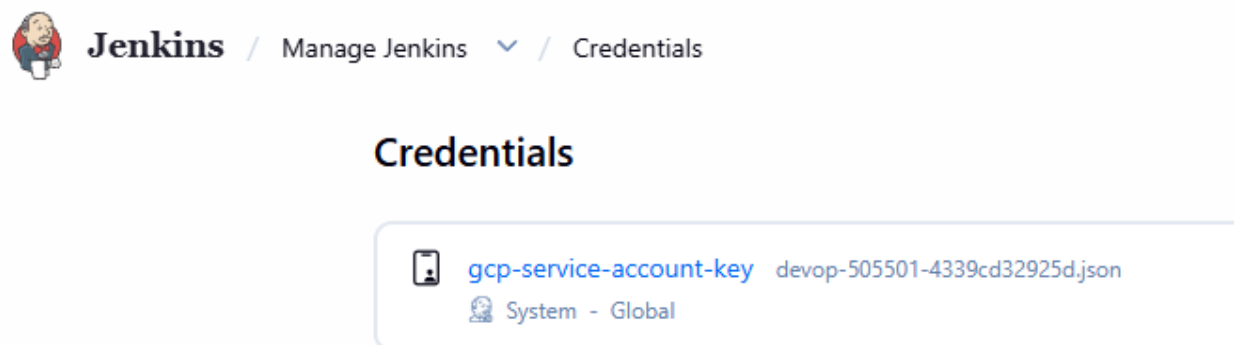


Figure 9: Credencial `gcp-service-account-key` en Jenkins

Stage 1 — Checkout: Jenkins obtiene el Jenkinsfile desde GitHub y clona el repositorio:

Console

```
Started by user admin-jenkins
Obtained Jenkinsfile from git https://github.com/santilp95/01_devops_cd_cd
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/deploy-cloud-run-pipe
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/deploy-cloud-run-pipe/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/santilp95/01_devops_cd_cd # timeout=10
Fetching upstream changes from https://github.com/santilp95/01_devops_cd_cd
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --tags --force --progress -- https://github.com/santilp95/01_devops_cd_cd +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 6cccd304bc111845cbf855f4514d05b71841fd1e (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 6cccd304bc111845cbf855f4514d05b71841fd1e # timeout=10
Commit message: "fix: update PROJECT_ID in Jenkinsfile to correct value"
> git rev-list --no-walk 6cccd304bc111845cbf855f4514d05b71841fd1e # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] timestamps
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] checkout
```

Figure 10: Consola: checkout del repositorio

Stage 2 — Build: construye la imagen Docker y la etiqueta con el número de build y latest:

```
11:26:14 Selected Git installation does not exist. Using Default
11:26:14 The recommended git tool is: NONE
11:26:14 No credentials specified
11:26:14 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/deploy-cloud-run-pipe/.git # timeout=10
11:26:14 Fetching changes from the remote Git repository
11:26:14 > git config remote.origin.url https://github.com/santilp95/01_devops_cd_cd # timeout=10
11:26:14 Fetching upstream changes from https://github.com/santilp95/01_devops_cd_cd
11:26:15 > git --version # timeout=10
11:26:15 > git --version # 'git version 2.47.3'
11:26:15 > git fetch --tags --force --progress -- https://github.com/santilp95/01_devops_cd_cd +refs/heads/*:refs/remotes/origin/* # timeout=10
11:26:15 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
11:26:15 Checking out Revision 6cced304bc11845cbf855f4514d05b71841fd1e (refs/remotes/origin/main)
11:26:15 > git config core.sparsecheckout # timeout=10
11:26:15 > git checkout -f 6cced304bc11845cbf855f4514d05b71841fd1e # timeout=10
11:26:15 Commit message: "fix: update PROJECT_ID in Jenkinsfile to correct value"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
11:26:15 + docker build -t us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:5 -t us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:latest .
11:26:15 DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
11:26:15     Install the buildx component to build images with BuildKit:
11:26:15     https://docs.docker.com/go/buildx/
11:26:15
11:26:15 Sending build context to Docker daemon 3.869MB
```

Figure 11: Consola: build de la imagen Docker

Stage 3 — Deploy (autenticación y push): activa la service account, configura el proyecto GCP y publica la imagen en Artifact Registry (el WARNING sobre la Cloud Resource Manager API es informativo, no detiene el pipeline):

Stage 3 — Deploy (gcloud run deploy): despliega la nueva revisión en Cloud Run, revoca las credenciales temporales y cierra el pipeline con Finished: SUCCESS:

Configuración necesaria antes de correr el Jenkinsfile

1. En GCP, crear una service account con estos roles:
 - roles/artifactregistry.writer (subir la imagen)
 - roles/run.admin (desplegar el servicio)
 - roles/iam.serviceAccountUser (actuar como la SA de ejecución de Cloud Run)
2. Generar y descargar la clave JSON de esa service account.
3. En Jenkins: **Manage Jenkins → Credentials** → agregar una credencial tipo *Secret file* con el contenido de la clave JSON:
 - ID: gcp-service-account-key (debe coincidir con el credentialsId del Jenkins-file).
4. Si el repositorio mi-app en Artifact Registry no existe todavía, créalo antes de correr el pipeline:

```
gcloud artifacts repositories create mi-app \
  --repository-format=docker \
  --location=us-central1
```
5. Asegurarse de que el agente de Jenkins tenga **Docker** y el **SDK de gcloud** instalados, y que el usuario jenkins tenga permisos para usar Docker (usermod -aG docker jenkins).


```

11:26:15 Successfully built 9a37da581a21
11:26:16 Successfully tagged us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:5
11:26:16 Successfully tagged us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:latest
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] withCredentials
11:26:16 Masking supported pattern matches of $GCP_KEY_FILE
[Pipeline] {
[Pipeline] sh
11:26:16 + gcloud auth activate-service-account --key-file=****
11:26:17 Activated service account credentials for: [github-actions-sa@devop-505501.iam.gserviceaccount.com]
11:26:17 + gcloud config set project devop-505501
11:26:19 WARNING: [github-actions-sa@devop-505501.iam.gserviceaccount.com] does not have permission to access projects instance [devop-505501] (or it may not exist): Cloud Resource Manager API has not been used in project 261835461000 before or it is disabled. Enable it by
visiting https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview/project-261835461000 then retry. If you enabled this API recently, wait a few minutes for the action to propagate to our systems and retry. This command is authenticated as
github-actions-sa@devop-505501.iam.gserviceaccount.com which is the active account specified by the [core/account] property.
11:26:19 - '@type': type.googleapis.com/google.rpc.ErrorInfo
11:26:19 domain: googleapis.com
11:26:19 metadata:
11:26:19 activationUrl: https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview/project-261835461000
11:26:19 consumer: projects/261835461000
11:26:19 containerInfo: '261835461000'
11:26:19 service: cloudresourcemanager.googleapis.com
11:26:19 serviceTitle: Cloud Resource Manager API
11:26:19 reason: SERVICE_DISABLED
11:26:19 - '@type': type.googleapis.com/google.rpc.LocalizedMessage
11:26:19 locale: en-US
11:26:19 message: Cloud Resource Manager API has not been used in project 261835461000 before
or it is disabled. Enable it by visiting https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview/project-261835461000
then retry. If you enabled this API recently, wait a few minutes for the action
to propagate to our systems and retry.
11:26:19 - '@type': type.googleapis.com/google.rpc.Help
11:26:19 links:
11:26:19 - description: Google developers console API activation
11:26:19 url: https://console.developers.google.com/apis/api/cloudresourcemanager.googleapis.com/overview/project-261835461000
11:26:19 Updated property [core/project].
11:26:19 + gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
11:26:19 Adding credentials for: us-central1-docker.pkg.dev
11:26:19 Docker configuration file updated.
11:26:19 + docker push us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:5
11:26:20 The push refers to repository [us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app]

```

Figure 12: Consola: autenticación en GCP y push de la imagen

```

11:26:25 latest: digest: sha256:9a37da581a211b74e532336cd8245e56658749e4f7e7a1424a47f83bcd3db27 size: 2056
11:26:25 + gcloud run deploy mi-app --image=us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:5 --region=us-central1 --platform-managed --quiet
11:26:27 Deploying container to Cloud Run service [mi-app] in project [devop-505501] region [us-central1]
11:26:27 Deploying...
11:26:34 Creating Revision.....done
11:26:34 Routing traffic.....done
11:26:35 Done.
11:26:36 Service [mi-app] revision [mi-app-00005-f92] has been deployed and is serving 100 percent of traffic.
11:26:36 Service URL: https://mi-app-261835461000.us-central1.run.app
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
11:26:36 + gcloud auth revoke --all --quiet
11:26:37 WARNING: [github-actions-sa@devop-505501.iam.gserviceaccount.com] appears to be a service account. Service account tokens cannot be revoked, but they will expire automatically. To prevent use of the service account token earlier than the expiration, delete or
disable the parent service account. To explicitly delete the key associated with the service account use 'gcloud iam service-accounts keys delete' instead.
11:26:37 Revoked credentials:
11:26:37 - github-actions-sa@devop-505501.iam.gserviceaccount.com
[Pipeline] echo
11:26:37 Deploy completado: us-central1-docker.pkg.dev/devop-505501/mi-app/mi-app:5 publicado y desplegado en Cloud Run (mi-app).
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Figure 13: Consola: despliegue en Cloud Run y cierre del pipeline

GCP

Resultado final del despliegue: el proyecto de Google Cloud usado por ambos pipelines (devop-505501) y la comprobación de que la app desplegada en Cloud Run responde correctamente.

Proyecto en Google Cloud Console (01 devop, project ID devop-505501), donde vive el servicio de Cloud Run, Artifact Registry y las service accounts usadas por CI/CD:

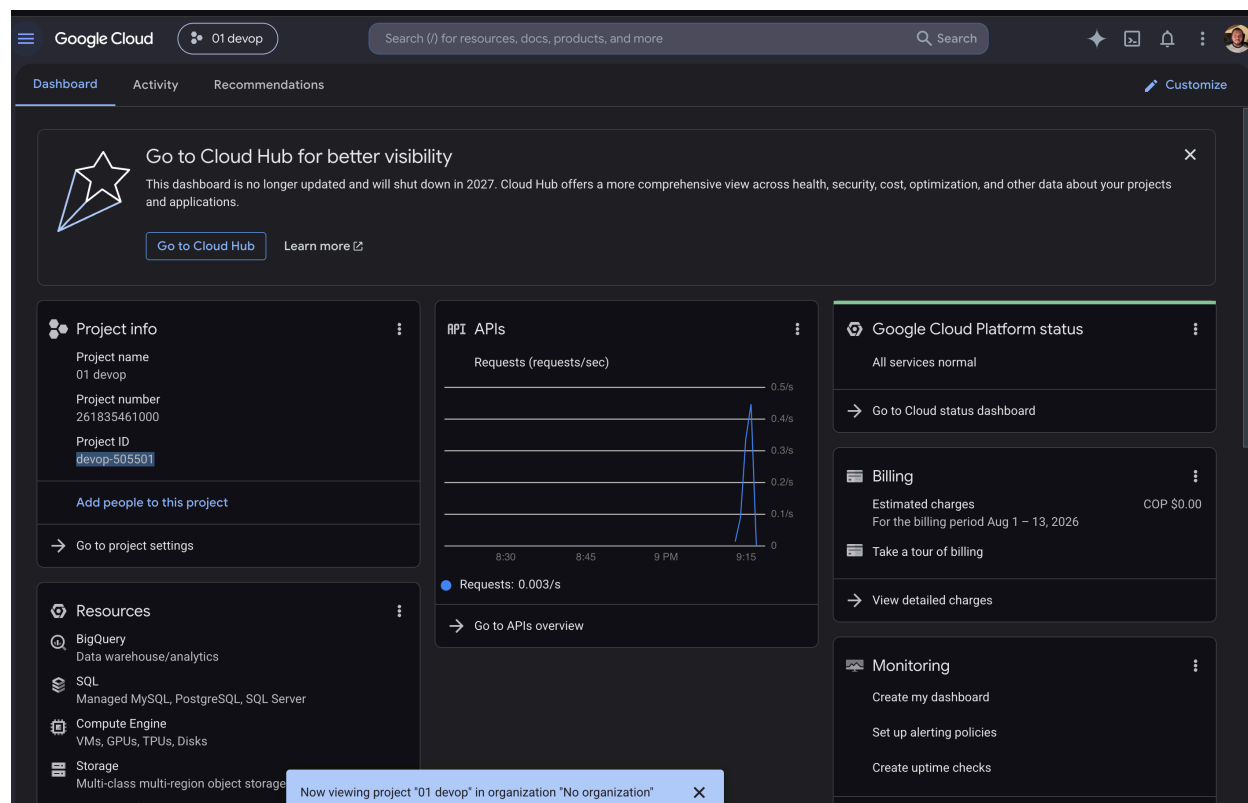


Figure 14: Dashboard del proyecto en Google Cloud

Prueba del API ya desplegado: petición a `https://mi-app-261835461000.us-central1.run.app/health` respondiendo 200 OK con `{"status":"ok"}`, confirmando que el deploy (tanto por GitHub Actions como por Jenkins) queda funcionando en producción:

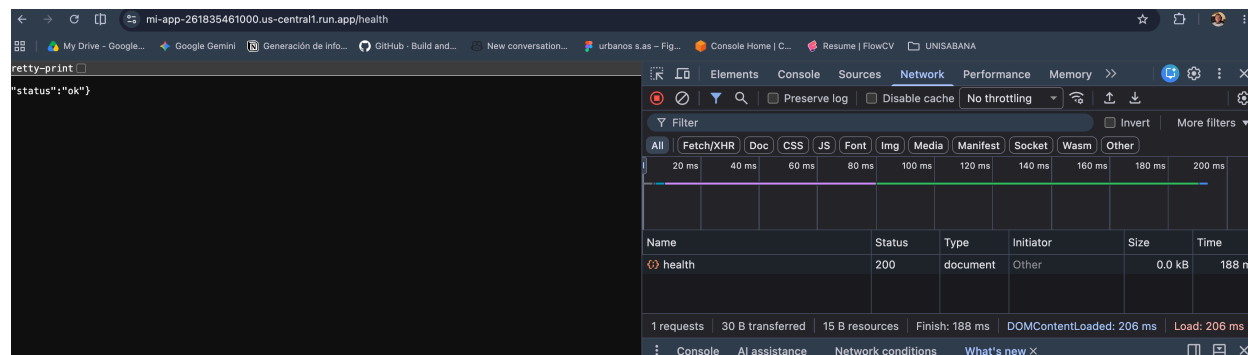


Figure 15: Prueba del endpoint /health en Cloud Run

También se puede probar directo desde la terminal:

```
curl https://mi-app-261835461000.us-central1.run.app/health
```

Estructura del repositorio

```
.
├── .github/workflows/ci.yml          # Pipeline CI (GitHub Actions)
├── .github/workflows/deploy.yml     # Pipeline CD a Cloud Run (GitHub Actions)
├── Jenkinsfile                     # Pipeline CD a Cloud Run (Jenkins)
├── Dockerfile                      # Multi-stage: build TS → imagen de producción
├── tsconfig.json
├── jest.config.js
├── src/index.ts                    # Código de la aplicación (TypeScript)
├── tests/app.test.ts               # Pruebas unitarias (Jest + ts-jest + Supertest)
└── package.json
```